

Minimum Genetic Mutation

© 2026 Arpit Raj. All rights reserved.

Key Approach

Use BFS because every mutation costs 1.

- Treat every gene string as a node in an implicit graph.
- Store all valid genes from bank in a set for $O(1)$ lookup.
- **Queue stores:** {current_gene, mutations_count}

Algorithm Steps

Start BFS from:

- {startGene, 0}

For every current gene:

- Try changing every character position.
- Replace it with one of: **A, C, G, T**
- Generate all possible next genes.

If generated gene:

- exists in bank
- is not already visited
- mark visited

→ push with count + 1

- If current gene becomes endGene, return mutations.
- If BFS ends without reaching target, return -1.

Pattern Summary

- BFS on implicit graph
- State = gene string
- Generate neighbours by changing one character
- Visited prevents repeated states

C++ Implementation

```
class Solution {
public:
    int minMutation(string startGene, string endGene, vector<string>& bank) {
        unordered_set<string> st;
        unordered_set<string> seen;
        for(string s:bank){
            st.insert(s);
        }

        vector<char> choices={'A','C','G','T'};
        queue<pair<string,int>> q;

        //kis string ko bananae me kitne transform

        q.push({startGene,0});
        seen.insert(startGene);

        while(!q.empty()){
            string currs=q.front().first;
            int currd=q.front().second;
            q.pop();
            if(currs==endGene) return currd;

            for(int i=0;i<currs.size();i++){
                string news=currs;
                char orig=news[i];
                for(char c:choices){
                    if(c==orig) continue;
                    news[i]=c;
                    if(seen.find(news)==seen.end() && st.find(news)!=st.end()){
                        seen.insert(news);
                        q.push({news,currd+1});
                    }
                }
            }
        }

        return -1;
    }
};
```